Techniques de Développement des Systèmes

Faculté des Sciences, de Génie et d'Architecture (FSGA)

Mini projet de fin de session

Système e-commerce multi tenant

Identification Étudiant	
Code Etudiant :	**SE185146**
Nom :	**SÉNATUS**
Prénom(s) :	**Natanael Godson Ralph**
Faculté :	**FSGA**
Option :	**Génie Informatique**

Février 2026

Système e-commerce multi tenant

A. Identification

A.1. Identification du document

Nom du document	Document de conception
Project:	Système e-commerce multi tenant
Version du projet:	1.0
Version du document:	1.1
Sécurité du document:	Privé
Date de création:	04/02/2026
Par:	N. Godson Ralph SÉNATUS

A.2. Historique des changements

Qui?	Quand?	Quoi?
N. Godson Ralph SÉNATUS	04/02/2026	Création du document
N. Godson Ralph SÉNATUS	06/02/2026	Mise à jour du document

A.3. Validation

Version	Nom	Fonction	Date	Signature	Commentaires
V 1.0	N. Godson Ralph SÉNATUS	Chef de projet	04/02/26		
V 1.1	N. Godson Ralph SÉNATUS	Chef de projet	06/02/26		

Système e-commerce multi tenant

Table of Contents

A.1. Identification du document *2*
A.2. Historique des changements 2
A.3. Validation 2
1. Objectif du Document 4
2. Presentation Generale du Système 4
3. Cas d'utilisation Fonctionnels (Use Cases) 4
4. Diagrammes UML 5
5. Règles métiers globales 7
6. Structure de données Globales 7
7. Contraintes Techniques à respecter 8
8. Annexes 8

Système e-commerce multi tenant

1. Objectif du Document

Ce document a pour vocation de transformer les besoins exprimés dans le cahier des charges en une vision claire et structurée du système à développer. Il vise à décrire de manière détaillée les choix d'architecture, les principes de conception ainsi que les composants essentiels de la solution proposée. À travers ce document, les fonctionnalités du système, les interactions entre les différents acteurs et les règles de fonctionnement sont présentées afin de faciliter la compréhension globale du projet. Il constitue ainsi une référence technique destinée à guider le développement, à assurer la cohérence des implémentations et à garantir le respect des exigences fonctionnelles et non fonctionnelles.

2. Presentation Generale du Système

Le système proposé est une plateforme e-commerce multi-tenant de type SaaS permettant à plusieurs boutiques indépendantes de coexister au sein d'un même environnement. Chaque propriétaire de boutique dispose d'un espace sécurisé pour gérer ses produits, suivre ses ventes et administrer son catalogue, tandis que les utilisateurs peuvent créer un compte, parcourir les différentes offres, ajouter des articles à leur panier et effectuer des achats. Le système vise à simplifier la gestion des ventes en ligne tout en offrant une expérience fluide aux clients. Il intègre des fonctionnalités telles que la gestion des utilisateurs, le panier de commande et des options de paiement simulées, garantissant ainsi un processus d'achat complet. Cette solution répond au besoin d'un outil centralisé, fiable et évolutif pour la vente de produits en ligne.

3. Cas d'utilisation Fonctionnels (Use Cases)

Cas d'utilisation 1 : Création de boutique

Ce cas d'utilisation permet à un utilisateur de créer une nouvelle boutique sur la plateforme. Lors de cette opération, le système enregistre les informations de base de la boutique et lui attribue un identifiant unique. Cette étape constitue le point de départ pour qu'un tenant puisse commencer à proposer ses produits en ligne.

Règles métiers:

- Le nom de la boutique doit être obligatoire et unique.
- Chaque boutique est associée à un identifiant (tenant_id).
- Une boutique créée est automatiquement active.
- Une boutique ne peut être créée que par un utilisateur autorisé.

Système e-commerce multi tenant

Interfaces à développer:

- Formulaire de création de boutique.
- Message de confirmation ou d'erreur.

Cas d'utilisation 2 : Création de compte utilisateur

Ce cas d'utilisation permet à un utilisateur de s'inscrire sur la plateforme en fournissant ses informations personnelles. Le système crée alors un profil utilisateur qui pourra être utilisé pour effectuer des achats ou gérer une boutique selon son rôle.

Règles métiers:

- L'adresse courriel doit être unique.
- Tous les champs obligatoires doivent être renseignés.
- Chaque utilisateur est associé à un tenant.
- Le rôle de l'utilisateur doit être défini (CLIENT ou TENANT).

Interfaces à développer:

- Page d'inscription.
- Message de succès ou d'erreur.

Cas d'utilisation 3 : Gestion des produits

Ce cas d'utilisation permet au tenant d'ajouter, modifier, supprimer et consulter les produits de sa boutique. Le système enregistre chaque produit avec ses informations essentielles telles que le nom, le prix et la quantité en stock.

Règles métiers:

- Un produit appartient obligatoirement à une boutique.
- Le prix d'un produit doit être supérieur à zéro.
- Le stock ne peut pas être négatif.
- Un tenant ne peut gérer que les produits de sa propre boutique.

Interfaces à développer:

- Formulaire d'ajout et de modification de produit.
- Liste des produits.

Cas d'utilisation 4 : Ajout au panier

Ce cas d'utilisation permet à un utilisateur d'ajouter un ou plusieurs produits à son panier afin de préparer une commande.

Règles métiers:

- Un utilisateur doit être identifié pour utiliser le panier.
- La quantité ajoutée doit être supérieure à zéro.
- Un produit ajouté doit exister dans le catalogue.

Interfaces à développer:

- Bouton « Ajouter au panier ».
- Page panier.

Cas d'utilisation 5 : Paiement

Ce cas d'utilisation permet à l'utilisateur de valider son panier et de choisir un mode de paiement afin de finaliser sa commande.

Règles métiers:

- Le panier ne doit pas être vide.
- Le système propose plusieurs modes de paiement (wallet, carte de crédit, carte de débit).
- Une commande validée est enregistrée avec le statut « payée ».

Interfaces à développer:

- Page de paiement.
- Message de confirmation.

4. Diagrammes UML

4.1. Diagramme de **Cas d'Utilisation**
4.2. Diagramme de **Séquence**

4.3. Diagramme **d'État-Transition**

5. Règles métiers globales

- Chaque boutique est identifiée par un tenant_id unique.
- Un utilisateur ne peut accéder qu'aux données correspondant à sa boutique.
- Les produits doivent toujours être associés à une boutique valide.
- Les opérations de création utilisent la méthode POST.
- Les données doivent être validées avant insertion dans la base.
- Un utilisateur doit exister avant de pouvoir ajouter des produits au panier.
- Une commande validée ne peut pas être modifiée.
- Le système doit garantir l'intégrité des données via des clés étrangères.

6. Structure de données Globales

Cette section présente les principales structures de données nécessaires à l'implémentation des fonctionnalités du système e-commerce multi-tenant. Chaque structure correspond à une entité essentielle du système et regroupe les informations indispensables à son fonctionnement.

Boutique (Tenant)

- Id
- Nom
- Date_creation

Utilisateur (User)

- Id
- Tenant_id
- Nom
- Courriel
- Mot_de_passe
- Rôle (CLIENT, TENANT)

Produit (Product)

- Id
- Tenant_id
- Nom
- Prix
- Stock

Panier (Cart)

- Id
- User_id
- Product_id
- Quantité

Commande (Order)

- Id
- User_id
- Tenant_id
- Total
- Méthode_paiement (Wallet, Carte crédit, Carte débit)
- Statut
- Date_creation

7. Contraintes Techniques à respecter

- Utilisation d'une architecture 3-tiers combinée au pattern MVC.
- Base de données partagée avec identification des boutiques via tenant_id.
- Respect des principes REST pour la communication entre frontend et backend.
- Validation des données côté serveur avant toute insertion en base.
- Séparation claire entre logique métier, accès aux données et présentation.
- Compatibilité avec les navigateurs web courants.
- Sécurisation minimale des accès et des échanges.
- Code structuré et commenté afin de faciliter la maintenance.

8. Annexes

- **Tenant** : Propriétaire d'une boutique sur la plateforme.
- **Utilisateur** : Personne utilisant le système pour effectuer des achats.
- **Produit** : Article proposé à la vente par une boutique.
- **Panier** : Espace temporaire contenant les produits sélectionnés.
- **Commande** : Ensemble des produits validés pour achat.
- **Paiement simulé** : Processus de validation sans intégration réelle de passerelle bancaire.
- **Multi-tenant** : Architecture permettant à plusieurs boutiques de partager la même application tout en conservant leurs données isolées.
- **REST** : Representational State Transfer
- **MVC** : Model-View-Controller